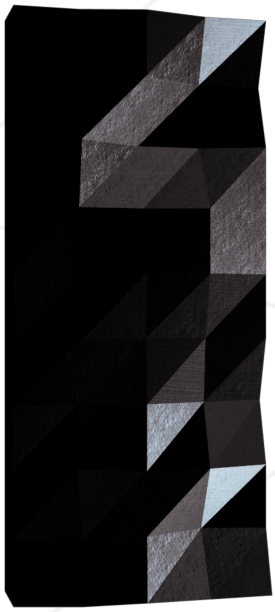


RAG against the machine

Will you answer my questions?

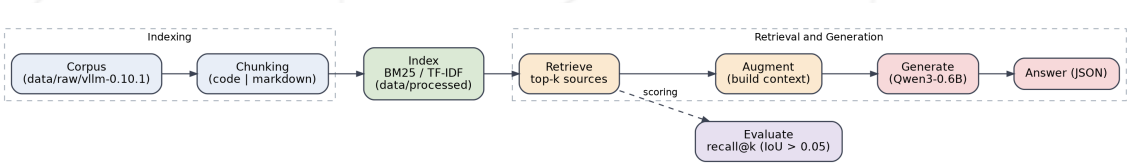
Summary: *Retrieval-Augmented Generation, that's it.*
That's the goal of this project.

Version: 2.0

Contents

Chapter I

Preamble



The pipeline you will build: ingest and index a codebase, retrieve the most relevant snippets for a question, hand them to a small local model to generate a grounded answer, and measure retrieval quality with recall@k.

Chapter II

Foreword

A language model is frozen in time. Everything it "knows" was fixed the day its training ended. Ask it about a library released last month, a private codebase, or yesterday's incident report, and it will either shrug or invent a confident answer out of thin air.

Retraining a model every time the world changes is absurdly expensive, so we cheat. Instead of stuffing more knowledge *into* the model, we let it *reach out* for the right information at the moment it answers, the same way you don't memorise a whole manual but you know which page to open.

That sounds easy until you try it. The "manual" here is an entire codebase: thousands of files, hundreds of thousands of lines. Finding the two or three snippets that actually answer a question, and *only* those, is a needle-in-a-haystack problem. Search too little and you miss the answer; search too much and you drown the model in noise.

Our intuitions are bad at this kind of scale, but good engineering is not. Retrieving the right evidence from a mountain of data, then making a small model answer faithfully from it, is what this project is about.

Chapter III

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills — both coding and non-coding — that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.

- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter IV

Introduction

We met **function calling** in *call_me_maybe*; this time the topic is **RAG**. Before looking at what RAG *is*, focus on what it *does*.

A model only "knows" what it was trained on, and retraining it to add knowledge is slow and costly. RAG takes another route: instead of putting knowledge *into* the model, you give it access to an external source of *your* choosing and let it pull from there at answer time.

In practice, RAG has four stages:

- **Indexing**: organise the data so it can be searched.
- **Retrieving**: match a question against the index and pull the most relevant snippets.
- **Augmenting**: filter those snippets and place them in the model's context window.
- **Generating**: read that context and produce the answer.

`Read the whole document before writing any code: each stage feeds the next.`

Chapter V

Common Instructions

V.1 General Rules

- Your project must be written in **Python 3.10 or later**.
- Your project must adhere to the **flake8** coding standard.
- Your functions should handle exceptions gracefully to avoid crashes. Use `try-except` blocks to manage potential errors. Prefer context managers for resources like files or connections to ensure automatic cleanup. If your program crashes due to unhandled exceptions during the review, it will be considered non-functional.
- All resources (e.g., file handles, network connections) must be properly managed to prevent leaks. Use context managers where possible for automatic handling.
- Your code must include type hints for function parameters, return types, and variables where applicable (using the `typing` module). Use `mypy` for static type checking. All functions must pass mypy without errors.
- Include docstrings in functions and classes following PEP 257 (e.g., Google or NumPy style) to document purpose, parameters, and returns.

V.2 Makefile

Include a `Makefile` in your project to automate common tasks. It must contain the following rules (mandatory lint implies the specified flags; it is strongly recommended to try `-strict` for enhanced checking):

- **install**: Install project dependencies using `pip`, `uv`, `pipx`, or any other package manager of your choice.
- **run**: Execute the main script of your project (e.g., via Python interpreter).
- **debug**: Run the main script in debug mode using Python's built-in debugger (e.g., pdb).

- **clean**: Remove temporary files or caches (e.g., __pycache__, .mypy_cache) to keep the project environment clean.
- **lint**: Execute the commands `flake8 .` and `mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs`
- **lint-strict** (optional): Execute the commands `flake8 .` and `mypy . --strict`

V.3 Additional Guidelines

- Create test programs to verify project functionality (not submitted or graded). Use frameworks like `pytest` or `unittest` for unit tests, covering edge cases.
- Include a `.gitignore` file to exclude Python artifacts.
- It is recommended to use virtual environments (e.g., venv or conda) for dependency isolation during development.

If any additional project-specific requirements apply, they will be stated immediately below this section.

V.4 Additional Requirements

In addition to the common Python rules above, this project requires:

- Your **data models** must use `pydantic` for validation and type safety: the structures you exchange between stages, such as those in the Data Models section. Service or orchestration classes (indexer, retriever, pipeline, etc.) do not have to be pydantic.
- The **flake8** standard applies to bonus files as well.
- You must use `uv` as your project and package manager: the reviewer and the moulinette only run `uv sync`.
- Your system must expose a Command-Line Interface (CLI) built with **Python Fire**, and progress bars (`tqdm`) for long-running operations.
- You need to use the following model:
  - **Qwen/Qwen3-0.6B** (default)
  - You may use other models as long as everything still works with **Qwen/Qwen3-0.6B**.
- Beyond the tools required above, you may use any library you like.

Chapter VI

Mandatory part

You will build a **Retrieval-Augmented Generation system** that answers questions about a codebase. You ingest the provided vLLM repository into a searchable index, retrieve the most relevant snippets for a question, generate an answer from them with `Qwen/Qwen3-0.6B`, and measure retrieval quality with recall@k. Your system is judged on whether it retrieves the right source locations and produces answers grounded in them.

VI.1 Indexing the codebase

Everything starts with the index. Read the files you judge useful from the vLLM repository shipped in the attachments, split each one into chunks, and persist an index that retrieval can query in milliseconds. Indexing the whole corpus must take **at most 5 minutes**.

A Python file and a Markdown page do not break apart the same way, so your program must implement **two distinct chunking strategies**:

- Python code chunking,
- Markdown / text chunking.

For retrieval itself, implement **at least one** of the two classic lexical methods. The choice is yours:

- TF-IDF,
- BM25.

You may explore other methods on top, as long as one of these two is implemented.

Chunk size is configurable through a CLI argument (`--max_chunk_size`), with a default of 2000 characters. Do not go above it: the moulinette rejects any retrieved source longer than 2000 characters (its `max_context_length`), and a single over-long source makes your whole output invalid. Smaller chunks are fine; report the effect on your recall@k.

```
Terminal - indexing

wil@42:~/rag$ uv run python -m src index --max_chunk_size 2000
Chunking:   100%|##########| 1965/1965 [00:00<00:00, 16710 file/s]
Tokenizing: 100%|##########| 13466/13466 [00:01<00:00, 10668 chunk/s]
Ingestion complete! Indexed 13466 chunks under data/processed/
```

Indexing a target corpus: the indexer chunks every file and persists the index under `data/processed/`.

Questions and source code rarely use the same words: a question may paraphrase an idea or quote an identifier verbatim. What you keep at indexing time decides which of the two you can still match.

VI.2 Retrieval

With the index built, you can search it. Given a question, your system returns the **top-k** most relevant snippets. Each result is a source location: a `file_path` and the character range (`first_character_index`, `last_character_index`) it covers, at most **2000 characters** wide.

`file_path` must match the corpus path **exactly** (e.g. `data/raw/vllm-0.10.1/docs/features/lora.md`). The grader compares paths verbatim, so a path with a different prefix never matches.

Retrieval must work for a single query and in batch over a whole dataset of questions read from JSON. On the reference datasets, your system must reach at least **80 % recall@5 on docs questions** and **50 % on code questions** (the metric is defined in the Evaluation chapter).

```
Terminal - search
wil@42:~/rag$ uv run python -m src search "How to configure the OpenAI server?" --k 5
data/raw/vllm-0.10.1/examples/online_serving/openai_transcription_client.py [0:1352]
data/raw/vllm-0.10.1/examples/online_serving/prompt_embed_inference_with_openai_client.py [0:932]
data/raw/vllm-0.10.1/docs/deployment/frameworks/dstack.md [1936:3170]
data/raw/vllm-0.10.1/docs/models/supported_models.md [1699:3396]
data/raw/vllm-0.10.1/examples/online_serving/openai_chat_completion_client_with_tools.py [0:2000]
```

A single-query search returns ranked source locations, each with its file path and character span.

VI.3 Answer generation

With the right snippets retrieved, the system generates a natural-language answer using `Qwen/Qwen3-0.6B`. Pass the retrieved context to the model within its token budget, and produce structured JSON following the provided pydantic models.

A satisfactory answer is:

- **Coherent** and understandable,
- **Grounded** in the retrieved sources, with no major hallucination,
- **On point**: it answers the question actually asked.

The mandatory Qwen/Qwen3-0.6B model has known reasoning limits. These criteria describe the target an answer should aim for, not a strict pass/fail bar: grading prioritizes retrieval quality, grounding and prompt strategy over perfect final phrasing. A valid answer is coherent and mostly grounded in the retrieved documents, even if partially incomplete due to base-model limitations.

VI.4 Data Models

Implement the following `pydantic` models; they validate the data exchanged between the stages. The **MinimalSource** model represents a single source of information:

MinimalSource Model

```
class MinimalSource(BaseModel):
    file_path: str
    first_character_index: int
    last_character_index: int
```

The **UnansweredQuestion** and **AnsweredQuestion** models represent an unanswered question and an answered question:

UnansweredQuestion and AnsweredQuestion Models

```
class UnansweredQuestion(BaseModel):
    question_id: str = Field(default_factory=lambda:
str(uuid.uuid4()))
    question: str

class AnsweredQuestion(UnansweredQuestion):
    sources: List[MinimalSource]
    answer: str
```

The **RagDataset** model represents a dataset of RAG questions:

RagDataset Model

```
class RagDataset(BaseModel):
    rag_questions: List[AnsweredQuestion | UnansweredQuestion]
```

The **MinimalSearchResults** and **MinimalAnswer** models represent the search results and an answer:

MinimalSearchResults and MinimalAnswer Models

```
class MinimalSearchResults(BaseModel):
    question_id: str
    question: str
    retrieved_sources: List[MinimalSource]

class MinimalAnswer(MinimalSearchResults):
    answer: str
```

The **StudentSearchResults** and **StudentSearchResultsAndAnswer** models represent search results and search results with answers:

StudentSearchResults and StudentSearchResultsAndAnswer Models

```
class StudentSearchResults(BaseModel):
    search_results: List[MinimalSearchResults]
    k: int

class StudentSearchResultsAndAnswer(BaseModel):
    search_results: List[MinimalAnswer]
    k: int
```

The provided models are a foundation. You can expand them by adding new models or extra fields (for example in the search results model) if your implementation requires it.

VI.5 Output

Each command writes a JSON file that conforms to the provided pydantic models:

- **For search operations**: Use `StudentSearchResults` model with:
  - `search_results`: List of `MinimalSearchResults` containing question_id, question and retrieved_sources
  - `k`: Number of results requested
- **For answer generation**: Use `StudentSearchResultsAndAnswer` model with:
  - `search_results`: List of `MinimalAnswer` containing question_id, question, retrieved_sources, and answer
  - `k`: Number of results requested
- **Source information**: Each `MinimalSource` contains:
  - `file_path`: path to the source file, relative to your project root and written exactly as in the ingested corpus (e.g. `data/raw/vllm-0.10.1/...`); it is compared verbatim to the reference
  - `first_character_index`: Starting character position
  - `last_character_index`: Ending character position

Output Format

The output must respect the minimal basis of the provided models but can be enhanced as follows:

Example: StudentSearchResults Output


```
"search_results": [
   {
       "question_id": "q1",
       "question": "How to configure OpenAI server?",
       "retrieved_sources": [
           {
               "file_path": "data/raw/vllm-0.10.1/docs/serving/openai_compatible_server.md",
               "first_character_index": 9867,
               "last_character_index": 10100
           },
           {
               "file_path": "data/raw/vllm-0.10.1/vllm/entrypoints/openai/api_server.py",
               "first_character_index": 267,
               "last_character_index": 400
           }
       ]
   }
],
"k": 10
```


For answers, the output should follow the `StudentSearchResultsAndAnswer` model:

Example: StudentSearchResultsAndAnswer Output


```
"search_results": [
   {
       "question_id": "q1",
       "question": "How to configure OpenAI server?",
       "retrieved_sources": [
           {
               "file_path": "data/raw/vllm-0.10.1/docs/serving/openai_compatible_server.md",
               "first_character_index": 9867,
               "last_character_index": 10100
           },
           {
               "file_path": "data/raw/vllm-0.10.1/vllm/entrypoints/openai/api_server.py",
               "first_character_index": 267,
               "last_character_index": 400
           }
       ],
       "answer": "To configure the OpenAI compatible server in vLLM..."
   }
],
"k": 10
```

VI.6 Command-Line Interface

Provide a CLI built with **Python Fire**. Every command is invoked as `uv run python -m src <command> [options]`. The following commands are required (options shown are the minimum; you may add your own):

- `index -max_chunk_size <int>`
  Ingest `data/raw/` and build the index under `data/processed/`.
- `search <query> -k <int>`
  Return the top-`k` sources for a single query.
- `search_dataset -dataset_path <path> -k <int> -save_directory <dir>`
  Run search over a whole dataset and write a `StudentSearchResults` JSON file.
- `answer <query> -k <int>`
  Answer a single query using the retrieved context.
- `answer_dataset -student_search_results_path <path> -save_directory <dir>`
  Generate answers for a dataset, producing a `StudentSearchResultsAndAnswer` JSON file.
- `evaluate -student_search_results_path <path> -dataset_path <path>`
  Report your own recall@k against a ground-truth dataset, for your own testing.

All input and output paths must be configurable CLI arguments and never hard-coded. Include progress bars (`tqdm`) for long-running operations, and handle degenerate inputs (empty query, nonsensical query, `k=0`, missing files, malformed JSON) gracefully. The CLI is tested with such edge cases and must never crash with an unhandled traceback.

The evaluate command is for your own iteration. The official recall@k used during the defense is computed by the provided moulinette executable, not by your code. Your solution must never import or call the moulinette.

VI.7 End-to-end walkthrough

This section shows the complete workflow, from raw documents to evaluation and answer generation, exactly as the reference exam scripts run it.

VI.7.1 Expected project layout for evaluation

During the defense, the retrieval pipeline is run end-to-end as a single automated flow (`index` → `search_dataset` → moulinette `evaluate_student_search_results`) using reference exam scripts. The individual commands remain available for debugging, but to make the automated run reproducible without manual restructuring, your repository must respect the following layout and conventions:

- `src/`: your Python module, runnable as `uv run python -m src <command>`.
- `pyproject.toml` and `uv.lock` at the repository root, so `uv sync` works from the root.
- `README.md` at the repository root.
- `data/raw/`: the indexed sources (the provided vLLM repository).
- `data/processed/`: the index produced by the `index` command.
- `data/datasets/UnansweredQuestions/` and `data/datasets/AnsweredQuestions/`: the question datasets.
- `data/output/search_results/<DatasetScope>/`: the output of `search_dataset`, scoped by dataset.
- `data/output/search_results_and_answer/<DatasetScope>/`: the output of `answer_dataset`, scoped by dataset.

All input and output paths (`--dataset_path`, `--save_directory`, `--student_search_results_path`) must be configurable CLI arguments and must never be hard-coded: the evaluator points them at the datasets and at dedicated output directories. If the structure or the command interface is not respected, the reference scripts cannot run and the corresponding checks are considered failed by design.

VI.7.2 Running the full pipeline

The pipeline is driven by four commands, in order: index the corpus, search a whole dataset, score the results with the moulinette, then generate answers. The `search` and `answer` single-query commands shown earlier behave the same way on one question at a time.

1. Index the corpus once:

```
uv run python -m src index --max_chunk_size 2000
Ingestion complete! Indices saved under data/processed/
```

2. Search a dataset. Always scope `--save_directory` by dataset (`UnansweredQuestions` or `AnsweredQuestions`): the public datasets share file names, so writing every run into the same folder would overwrite previous results.

```
uv run python -m src search_dataset
  --dataset_path data/datasets/UnansweredQuestions/dataset_docs_public.json
  --k 10
  --save_directory data/output/search_results/UnansweredQuestions
Saved student_search_results to data/output/search_results/UnansweredQuestions/dataset_docs_public.json
```

3. Score with the moulinette (rename `moulinette-ubuntu`/`-fedora` to `moulinette` first). The student results come first, the ground-truth `AnsweredQuestions` dataset second:

```
./moulinette evaluate_student_search_results
  data/output/search_results/UnansweredQuestions/dataset_docs_public.json
  data/datasets/AnsweredQuestions/dataset_docs_public.json
  --k 10 --max_context_length 2000
```

```
Student data is valid: True
Evaluation Results
========================================
Recall@1: 0.450 Recall@3: 0.590 Recall@5: 0.650 Recall@10: 0.720
```

These figures illustrate the output format only. They are **not** a reference score. The thresholds you must reach are defined in the Evaluation chapter.

4. Generate answers from the search results:

```
uv run python -m src answer_dataset
  --student_search_results_path data/output/search_results/UnansweredQuestions/dataset_docs_public.json
  --save_directory data/output/search_results_and_answer/UnansweredQuestions
Loaded 100 questions ... Processed 100 of 100 questions
Saved student_search_results_and_answer to .../UnansweredQuestions/dataset_docs_public.json
```

Chapter VII

Evaluation

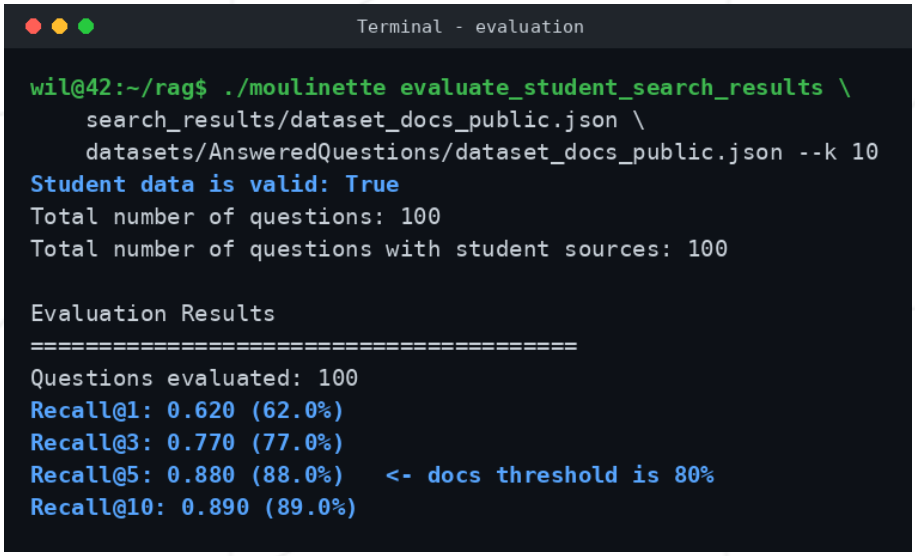
VII.1 Evaluation metrics

Retrieval quality is measured with a **recall@k** metric.

VII.1.1 Recall@k Calculation

For each question, recall@k is the share of its correct sources that you retrieve in your top-`k` results. A correct source counts as found when one of your results is in the *same file* and overlaps its character range.

The overlap bar is low (an IoU of `0.05`), so you do not need to match the reference span exactly: retrieving a chunk that covers the right region of the right file is enough. A result in a different file never counts, which is why `file_path` must be exact.



The moulinette validates your output, then reports recall at several values of **`k`**. *Docs must reach 80 % recall@5, code 50 %.*

VII.1.2 Performances

Your system must respect some minimal performances, listed below:

- **Indexing time**: at most 5 minutes for the whole corpus.
- **Retrieval throughput**: at most 90 seconds for 200 questions.
- **Recall@5**: at least **80% on docs questions** and **50% on code questions**.

Chapter VIII

Readme Requirements

A `README.md` file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.

The `README.md` must include at least:

- The very first line must be italicized and read: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- A "**Description**" section that clearly presents the project, including its goal and a brief overview.
- An "**Instructions**" section containing any relevant information about compilation, installation, and/or execution.
- A "**Resources**" section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the project.

➠ **Additional sections may be required depending on the project** (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

For this project, the `README.md` must also include:

- **System architecture**: Describe your RAG pipeline components and how they interact
- **Chunking strategy**: Explain your approach to document segmentation
- **Retrieval method**: Detail the retrieval algorithm and ranking mechanism

- **Performance analysis**: Discuss recall@k scores and system performance
- **Design decisions**: Explain key implementation choices
- **Challenges faced**: Document difficulties encountered and solutions
- **Example usage**: Provide clear examples of running your system

`Your README must be written in English.`

Chapter IX

Bonus Part

The bonus is graded only once the **entire mandatory part is validated**: every mandatory requirement met and the system robust even when a reviewer pushes it in odd directions. Until the mandatory part validates in full, the bonus is not evaluated at all.

There are **five** bonuses, each worth one point. They all run on a CPU-only campus machine, and each one extends the mandatory system in a direction you will meet again later. A bonus counts only if it is implemented and working, not merely described in the `README.md`, and you may be asked to demonstrate it.

1. **Semantic embeddings**: add a vector index built with a lightweight CPU model (such as `all-MiniLM-L6-v2`) next to your lexical index.
2. **Hybrid retrieval**: combine the lexical and semantic rankings into a single result list.
3. **Incremental indexing**: when a file changes, re-index only that file instead of rebuilding the whole index.
4. **Caching**: cache the index and query results to speed up cold start and repeated queries.
5. **Local HTTP API**: expose querying the index and answering questions over a small local HTTP API, so the system can be driven by something other than the CLI.

Chapter X

Submission and peer-evaluation

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double-check the names of your files to ensure they are correct.

Your repository must contain:

- `src/` directory with your implementation
- `pyproject.toml` and `uv.lock` for dependency management
- a `Makefile` exposing the rules from the Common Instructions (`install`, `run`, `debug`, `clean`, `lint`)
- `README.md` with comprehensive documentation
- Any additional configuration files needed to run your solution

`Do not include large data files, model weights, or generated outputs in your repository. The evaluator will generate these during the evaluation process. The deep-learning stack and the model weights can total several gigabytes, so build and run your project from a location with enough free disk space.`

X.1 Recode instructions

During the evaluation, a brief **modification of the project** may occasionally be requested. This could involve a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature.

While this step may **not be applicable to every project**, you must be prepared for it if it is mentioned in the evaluation guidelines.

This step is meant to verify your actual understanding of a specific part of the project. The modification can be performed in any development environment you choose (e.g., your usual setup), and it should be feasible within a few minutes — unless a specific time frame is defined as part of the evaluation.
You can, for example, be asked to make a small update to a function or script, modify a display, or adjust a data structure to store new information, etc.

The details (scope, target, etc.) will be specified in the **evaluation guidelines** and may vary from one evaluation to another for the same project.